Web Application Vulnerability Assessment & Penetration Testing (VAPT) Report

Target Application: Altoro Mutual (https://demo.testfire.net)
Assessment Type: Web Application & API Security Assessment
Assessor: Srikanth Yadav
Date: September 2026
Document Version: 2.0 (Comprehensive Technical Edition)

---

1. Executive Summary

A comprehensive Web Application Vulnerability Assessment and Penetration Testing (VAPT) engagement was executed on the **Altoro Mutual** web portal (demo.testfire.net). The objective was to perform a deep-dive security audit across client-side, server-side, authentication, session management, and API transaction layers.
The assessment uncovered critical system architectural flaws, including database layer manipulation (SQL Injection), arbitrary internal network requests (SSRF), state-changing request forgery (CSRF), parameter manipulation (IDOR), and client-side code execution (XSS). This report provides granular technical breakdowns, root-cause analyses, proof-of-concept mechanics, impact assessments, and enterprise remediation guidelines for each identified vulnerability.

Executive Risk & Vulnerability Matrix

Vulnerability Name	OWASP Category	CVSS v3.1 Score	Severity	Affected Endpoint
Authentication Bypass via Unsanitized Input	A03:2021 - Injection	9.8 (Critical)	**CRITICAL**	/login.jsp
Internal Service Enumeration & Egress Abuse	A10:2021 - SSRF	8.6 (High)	**HIGH**	/status.check.jsp

Vulnerability Name	OWASP Category	CVSS v3.1 Score	Severity	Affected Endpoint
Unauthenticated Session-Bound Fund Transfer	A01:2021 - Broken Access Control	8.1 (High)	**HIGH**	/bank/doTransfer
Reflected Context Script Execution	A03:2021 - Injection (XSS)	7.2 (High)	**HIGH**	/sendFeedback
Unauthorized Account Record Access	A01:2021 - Broken Access Control	6.5 (Medium)	**MEDIUM**	/bank/showAccount

---

2. Comprehensive Technical Vulnerability Analysis

2.1 Critical Severity: Administrative Authentication Bypass via SQL Injection (SQLi)

- **Target Endpoint:** https://demo.testfire.net/login.jsp
- **OWASP Top 10 Mapping:** A03:2021 - Injection
- **CVSS v3.1 Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H (Base Score: 9.8)

A. Technical Root Cause & Vulnerability Mechanism

The authentication interface concatenates raw, unsanitized user input directly into an active SQL query string executed against the backend database (e.g., Apache Derby or MySQL). When processing user logins, the backend constructs dynamic SQL statements similar to:
`SELECT * FROM users WHERE username = 'USER_INPUT' AND password = 'PASSWORD_INPUT'`
Because the application fails to utilize parameterized queries or binding variables, standard SQL control characters (such as single quotes ' and comment indicators -- or /*) break out of the intended data literal context and manipulate the SQL parser logic directly.

B. Exploitation Walkthrough & Proof of Concept (PoC)

1. The assessor navigated to the login portal at /login.jsp.
2. In the uid field, the classic boolean-based SQL injection payload was injected:

```
' OR 1=1--
```

3. The resulting query evaluated on the backend database as:
   ```
   SELECT * FROM users WHERE username = '' OR 1=1-- AND password =
   '...'
   ```
4. Because 1=1 continuously evaluates to **TRUE**, and the trailing double-dash (--) comments out the password verification logic, the database engine returned the first active user record in the database—which corresponds to the system Administrator.
5. **Resulting Access:** The session was granted full administrative privileges, redirecting to /bank/main.jsp with the welcome banner: *"Hello Admin User - Welcome to Altoro Mutual Online"*.

C. Business & Security Impact

- Total compromise of Confidentiality, Integrity, and Availability (CIA Triad).
- Unauthenticated attackers can bypass authentication controls, impersonate executive/admin users, inspect confidential banking data, alter balances, and potentially execute Remote Code Execution (RCE) via database extensions or UDFs depending on database privileges.

D. Actionable Remediation Guidance

- **Primary Fix - Parameterized Queries:** Replace all dynamic dynamic string queries with Prepared Statements.
  ```
  String query = "SELECT * FROM users WHERE username = ? AND
  password = ?";
  PreparedStatement pstmt = connection.prepareStatement(query);
  pstmt.setString(1, username);
  pstmt.setString(2, password);
  ResultSet results = pstmt.executeQuery();
  ```

- **Input Validation:** Implement restrictive whitelisting on login input fields to strip syntax control characters.

2.2 High Severity: Server-Side Request Forgery (SSRF)

- **Target Endpoint:** https://demo.testfire.net/status.check.jsp
- **OWASP Top 10 Mapping:** A10:2021 - Server-Side Request Forgery (SSRF)
- **CVSS v3.1 Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N (Base Score: 8.6)

A. Technical Root Cause & Vulnerability Mechanism

The "Server Status Check" module allows administrators and users to verify service availability. The backend application accepts a user-controlled URL or IP parameter and uses HTTP client libraries (e.g., Java HttpURLConnection) to initiate an HTTP request to the specified resource from the application server itself.
Due to the total absence of URL parsing restrictions, domain allowlists, or protocol filters, the backend blindly trusts the input parameter and executes outbound requests to arbitrary external destinations or internal network assets on behalf of the attacker.

B. Exploitation Walkthrough & Proof of Concept (PoC)

1. The assessor inspected the request sent to /status.check.jsp using Burp Suite Repeater.
2. The request parameter specifying the server status destination was modified to target loopback address spaces and internal services:
   ```
   GET /status.check.jsp?content=http://127.0.0.1:8080/admin HTTP/1.1
   Host: demo.testfire.net
   ```
3. The application server initiated a GET request to its own local port 8080 and reflected internal response metadata (HTTP status codes, server header banners, and body contents) back to the external client response.
4. **Burp Collaborator Verification:** Injecting an external out-of-band domain (e.g., http://[collaborator-id].oastify.com) confirmed that the application server initiates outbound HTTP connections directly.

C. Business & Security Impact

- Bypasses peripheral firewalls and Web Application Firewalls (WAF) to conduct internal port scanning and service discovery on the internal network segment.
- Can be leveraged to read metadata services in cloud environments (e.g., AWS Metadata Endpoint 169.254.169.254) to steal cloud IAM roles, storage keys, and deployment credentials.

D. Actionable Remediation Guidance

- **Network Domain Allowlists:** Implement a strict whitelist of approved external hosts/IPs that the application is allowed to query. Reject all requests outside the explicit allowlist.
- **Disable Loopback & Private IP Ranges:** Validate user input to explicitly block loopback addresses (127.0.0.1, localhost) and RFC 1918 private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16).
- **Network Isolation:** Segment the web application server into a DMZ with strict firewall rules prohibiting outbound connections to internal server subnets.

2.3 High Severity: Unauthorized Fund Transfer via Cross-Site Request Forgery (CSRF)

- **Target Endpoint:** https://demo.testfire.net/bank/doTransfer
- **OWASP Top 10 Mapping:** A01:2021 - Broken Access Control
- **CVSS v3.1 Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N (Base Score: 8.1)

A. Technical Root Cause & Vulnerability Mechanism

The fund transfer mechanism uses HTTP POST/GET requests to move monetary balances between specified bank accounts. However, the application relies solely on standard session cookies (JSESSIONID) for transaction authentication. The request lacks any unpredictable, session-bound anti-CSRF tokens or secondary confirmation steps.
When an authenticated user visits an external malicious website, the victim's browser automatically appends their active session cookies to cross-origin requests targeting demo.testfire.net, forcing the application to execute transfers without the user's explicit consent.

B. Exploitation Walkthrough & Proof of Concept (PoC)

1. An active authenticated session was established for checking account 800000.
2. The assessor intercepted a legitimate fund transfer request in Burp Suite:
```
POST /bank/doTransfer HTTP/1.1
Host: demo.testfire.net
Cookie: JSESSIONID=A1B2C3D4E5...

fromAccount=800000&toAccount=800001&transferAmount=500
```
3. A malicious HTML exploit page was hosted externally containing an auto-submitting form:
```
<form action="https://demo.testfire.net/bank/doTransfer"
method="POST" id="csrfForm">
  <input type="hidden" name="fromAccount" value="800000" />
  <input type="hidden" name="toAccount" value="800009" />
  <input type="hidden" name="transferAmount" value="10000" />
</form>
<script>document.getElementById('csrfForm').submit();</script>
```
4. Executing the PoC successfully transferred $10,000 from the victim's checking account 800000 into account 800009, yielding the response message: *"10000.0 was successfully transferred from Account 800000 into Account 800009"*.

C. Business & Security Impact

- Direct financial loss and fraud vector affecting banking customers.
- Attackers can execute unauthorized state-changing operations across user accounts including password changes, email modifications, and wire transfers.

D. Actionable Remediation Guidance

- **Synchronizer Token Pattern:** Generate unique, cryptographically secure anti-CSRF tokens for every user session and validate them on every state-changing POST/PUT request.
- **SameSite Cookie Attribute:** Enforce SameSite=Strict or SameSite=Lax on session cookies to restrict cross-site browser transmissions.
- **Re-Authentication:** Require users to enter their current password, PIN, or OTP before authorizing sensitive financial transactions.

2.4 High Severity: Reflected Cross-Site Scripting (XSS) via Fuzzing

- **Target Endpoint:** https://demo.testfire.net/sendFeedback
- **OWASP Top 10 Mapping:** A03:2021 - Injection (Cross-Site Scripting)
- **CVSS v3.1 Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N (Base Score: 7.2)

A. Technical Root Cause & Vulnerability Mechanism

The feedback submission endpoint accepts user feedback parameters (such as name, email address, subject, and comment body) and reflects them directly in the confirmation page. The

web application fails to perform proper context-aware HTML entity encoding or input filtering prior to rendering user-supplied strings in the DOM browser response.

B. Exploitation Walkthrough & Proof of Concept (PoC)

1. The assessor configured a automated fuzzing attack using **Burp Suite Intruder** against parameter inputs in the /sendFeedback endpoint.
2. Multiple XSS payload vectors were injected across parameters to test browser context handling. Key payloads tested included:
    - <script>alert(1)</script>
    - <body onload=alert(1)>
    - <input type="text" onfocus="alert(1)" autofocus>
3. Burp Intruder responses returned 200 OK status codes with verbatim reflecting payloads in the DOM.
4. Opening the resulting URL in a web browser triggered client-side JavaScript execution dialogs displaying false and alert pop-ups, confirming script context execution.

C. Business & Security Impact

- Attacker can hijack active user sessions by stealing JSESSIONID cookies.
- Enables keystroke logging, virtual defacement, or redirecting victims to malicious credential-harvesting phishing portals.

D. Actionable Remediation Guidance

- **Context-Aware Output Encoding:** Encode all dynamic content before rendering it in the HTML document using libraries like the OWASP Java Encoder (e.g., Encode.forHtml(userInput)).
- **Content Security Policy (CSP):** Deploy a strict CSP HTTP header to restrict inline script execution:
  `Header set Content-Security-Policy "default-src 'self'; script-src 'self';"`
- **HttpOnly Cookie Flag:** Mark all session cookies with the HttpOnly attribute to prevent access via JavaScript's document.cookie DOM API.

2.5 Medium Severity: Insecure Direct Object Reference (IDOR) & Parameter Manipulation

- **Target Endpoint:** https://demo.testfire.net/bank/showAccount?listAccounts=800001
- **OWASP Top 10 Mapping:** A01:2021 - Broken Access Control
- **CVSS v3.1 Vector:** CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N (Base Score: 6.5)

A. Technical Root Cause & Vulnerability Mechanism

The account selection module uses explicit database record identifiers in URL query parameters (e.g., listAccounts=800001) to retrieve historical ledger entries. The backend application retrieves and displays the requested records based purely on the parameter value supplied in the request, without validating whether the currently authenticated user is authorized to access that specific account number.

B. Exploitation Walkthrough & Proof of Concept (PoC)

1. The assessor logged in as a standard user assigned to account 800000.
2. The account summary URL was captured:
   `GET /bank/showAccount?listAccounts=800000 HTTP/1.1`
3. The parameter value was manually incremented to access an unlinked account:
   `GET /bank/showAccount?listAccounts=800001 HTTP/1.1`
4. The server responded with full transaction details, balance amounts ($70,000,000.00), deposits, checks, and paycheck transaction records belonging to account 800001.

C. Business & Security Impact

- Unauthorized disclosure of Personally Identifiable Information (PII) and confidential financial transaction records across customer accounts.
- Allows malicious actors to iterate through sequential account numbers to enumerate the entire user database.

D. Actionable Remediation Guidance

- **Indirect Reference Maps / Session Validation:** Store account authorizations in server-side user session objects. Validate that Session.getUserAccountIDs() contains the requested account ID prior to returning database records.
- **Use Cryptographically Secure Random Identifiers:** Replace predictable, incremental integer account identifiers with non-sequential UUIDs (e.g., f47ac10b-58cc-4372-a567-0e02b2c3d479).

---

3. Testing Methodology & Tooling Framework

- **Interception & Dynamic Analysis:** Burp Suite Professional / Community Edition (Proxy, Repeater, Intruder, Collaborator, Decoder).
- **API & Request Crafting:** Postman API Client.
- **Automated Fuzzing:** Burp Intruder Custom Payloads (OWASP ZAP / SecLists XSS & Injection dictionaries).
- **Methodology Alignment:** OWASP Web Security Testing Guide (WSTG v4.2) and Penetration Testing Execution Standard (PTES).

---

4. Comprehensive Remediation & Defense Roadmap

1. **Database Layer Hardening (Immediate Priority):** Convert all raw dynamic database queries across authentication and transaction modules into parameterized Prepared Statements.
2. **Session & Access Control Reinforcement:** Implement server-side session checks for all object access parameters (IDOR mitigation) and implement anti-CSRF token verification across state-changing endpoints.
3. **Input Sanitization & Output Encoding:** Apply contextual HTML encoding on dynamic text inputs to mitigate XSS vulnerabilities and enforce HttpOnly cookie flags.

4. **Egress Security & SSRF Protection:** Enforce strict egress URL whitelisting and block server requests directed to loopback addresses and private IP subnets.